LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 4&5

The Core of OOP & Encapsulation

Disusun Oleh:

Paulus Roberto Apolos

Siagian 105224020

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Sistem reservasi pada jaringan perhotelan saat ini rentan terhadap kendala operasional, seperti kelolosan jumlah tamu yang melebihi kapasitas (overcapacity) hingga penyalahgunaan kode voucher diskon. Jika dibiarkan, celah ini dapat mengganggu operasional dan merugikan manajemen secara finansial.

Proyek ini bertujuan merancang ulang arsitektur sistem reservasi (blueprint kelas KamarHotel) agar bertindak sebagai "pengawas" otomatis yang kebal terhadap kesalahan input (human error). Pendekatan utama yang digunakan berfokus pada dua pilar Object-Oriented Programming (OOP): Enkapsulasi dan Overloading Metode.

Melalui pembaruan ini, diharapkan operasional meja resepsionis perhotelan dapat berjalan jauh lebih presisi, aman, dan efisien.

II. Variabel

No	**Nama Variabel**	**Tipe Data**
1	nomorKamar	String
2	tipeKamar	String
3.	kapasitasMaksimal	Int
4.	hargaPerMalam	double
5.	isTersedia	Boolean
6.	jumlahTamu	Int
7.	jumlahMalam	Int
8.	kodeVoucher	String

9.	totalNormal	Double
10.	diskon	double
11.	kamar1, kamar2	boolean
12.	tagihanKamar1, tagihanKamar2	double

iii. **Constructor dan Method**

No	Nama Metode	Jenis Metode	Fungsi
1	KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal)	constructor	Membuat objek kamar baru dengan pendaftaran kilat. Harga otomatis diset 0 dan status kamar diset true (tersedia).
2	KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam)	constructor	Membuat objek kamar baru dengan pendaftaran lengkap beserta input harga per malam. Status kamar diset *true*.
3.	getNomorKamar(), getTipeKamar(), getKapasitasMaksimal(), getHargaPerMalam(), getIsTersedia()	Functional	Fungsinya sebagai getter untuk membaca/mengambil nilai dari masing masing atribut yang sudah dienkapsulasi.
4.	setTipeKamar(String)	procedural	Mengubah nilai tipe kamar dengan validasi bisnis yang ketat (hanya menerima input "Reguler", "Premium", atau "Suite").
5.	setHargaPerMalam(double)	procedural	Mengubah tarif harga per malam dengan validasi bisnis batas bawah (menolak angka di bawah 50.000).
6.	setNomorKamar(String), setKapasitasMaksimal(int)	procedural	Berfungsi sebagai pintu akses (Setter) untuk memodifikasi atau mengubah nilai atribut secara standar.
7.	batalPesan()	procedural	Membatalkan reservasi dengan mengubah status atribut isTersedia kembali menjadi true.
8.	pesanKamar()	Procedural	Melakukan pemesanan standar dengan mengubah status kamar menjadi tidak tersedia (false)
9.	pesanKamar(int jumlahTamu)	Procedural	Melakukan pemesanan dengan validasi ekstra untuk mengecek apakah jumlah tamu diinput melebihi kapasitas maksimal kamar (*overcapacity*).

10.	hitungTotalBayar(int jumlahMalam)	functional	Mengkalkulasi dan mengembalikan nilai total tagihan menginap dengan harga standar/normal.
11.	hitungTotalBayar(int jumlahMalam, string kodeVoucher)	Functional	Mengkalkulasi tagihan dengan validasi promo. Mengembalikan nilai yang sudah didiskon 20% jika syarat terpenuhi.
12.	tampilkanProfilKamar()	procedural	Menampilkan susunan cetak struk/profil yang berisi seluruh informasi detail atribut kamar ke layar komputer.

IV. Dokumentasi dan Pembahasan Code

Di bagian KamarHotel.java:

```java
public class KamarHotel {
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;

    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0;
        this.isTersedia = true;
    }

    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double harga) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(harga);
        this.isTersedia = true;
    }

    public String getNomorKamar() { return nomorKamar; }
    public String getTipeKamar() { return tipeKamar; }
    public int getKapasitasMaksimal() { return kapasitasMaksimal; }
    public double getHargaPerMalam() { return hargaPerMalam; }
    public boolean isTersedia() { return isTersedia; }

    public void setTipeKamar(String tipe) {
        if (tipe.equalsIgnoreCase("Reguler") || tipe.equalsIgnoreCase("Premium") || tipe.equalsIgnoreCase("Suite")) {
            this.tipeKamar = tipe;
        } else {
            System.out.println("Peringatan: Tipe '" + tipe + "' tidak valid. Otomatis diatur ke Reguler.");
            this.tipeKamar = "Reguler";
        }
    }

    public void setHargaPerMalam(double harga) {
        if (harga < 50000) {
            System.out.println("Peringatan: Harga di bawah 50000 ditolak. Otomatis diatur ke 50000.");
            this.hargaPerMalam = 50000;
        } else {
            this.hargaPerMalam = harga;
        }
    }

    public void pesanKamar() {
        if (isTersedia) {
            isTersedia = false;
            System.out.println("Kamar " + nomorKamar + " berhasil dipesan.");
        } else {
            System.out.println("Peringatan: Kamar " + nomorKamar + " sudah terisi.");
        }
    }

    public void pesanKamar(int jumlahTamu) {
        if (!isTersedia) {
            System.out.println("Peringatan: Kamar sudah terisi.");
        } else if (jumlahTamu > kapasitasMaksimal) {
            System.out.println("Ditolak: Jumlah tamu (" + jumlahTamu + ") melebihi kapasitas (" + kapasitasMaksimal + ")!");
        } else {
            isTersedia = false;
            System.out.println("Pemesanan berhasil untuk " + jumlahTamu + " orang.");
        }
    }

    public void batalPesan() {
        this.isTersedia = true;
    }

    public double hitungTotalBayar(int jumlahMalam) {
        return hargaPerMalam * jumlahMalam;
    }

    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
        double total = hitungTotalBayar(jumlahMalam);
        if (kodeVoucher.equalsIgnoreCase("PROMO") && jumlahMalam >= 3) {
            System.out.println("Voucher Berhasil: Diskon 20% diterapkan.");
            return total * 0.8;
        } else {
            System.out.println("Voucher Gagal: Syarat tidak terpenuhi atau kode salah. Harga normal diterapkan.");
            return total;
        }
    }

    public void tampilkanProfil() {
        System.out.println("=== Profil Kamar " + nomorKamar + " ===");
        System.out.println("Tipe: " + tipeKamar);
        System.out.println("Kapasitas: " + kapasitasMaksimal + " orang");
        System.out.println("Harga/Malam: Rp" + hargaPerMalam);
        System.out.println("Status: " + (isTersedia ? "Tersedia" : "Dipesan"));
        System.out.println("--------------------------");
    }
}
```

Di bagian Main.java:

```java
public class Main {
    Run | Debug
    public static void main(String[] args) {
        KamarHotel kamar1 = new KamarHotel(nomorKamar: "101", tipeKamar: "Presidential", kapasitasMaksimal: 2, -10000);
        KamarHotel kamar2 = new KamarHotel(nomorKamar: "102", tipeKamar: "Premium", kapasitasMaksimal: 2);
        kamar2.setHargaPerMalam(harga: 100000);

        System.out.println("\n--- Simulasi Operasional ---");
        System.out.println("Tagihan Kamar 1 (2 malam, PROMO): " + kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO"));
        kamar2.pesanKamar(jumlahTamu: 4);
        kamar2.pesanKamar(jumlahTamu: 2);
        kamar2.pesanKamar();
        System.out.println("Tagihan Kamar 2 (4 malam, PROMO): " + kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO"));

        System.out.println("\n--- Struk Informasi Akhir ---");
        kamar1.tampilkanProfil();
        kamar2.tampilkanProfil();
    }
}
```

Pembahasan Kode

```java
public class KamarHotel {
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;
```

Sistem ini dimulai dengan pembuatan *blueprint* kelas KamarHotel yang menerapkan prinsip Encapsulation ketat, di mana atribut seperti nomorKamar, tipeKamar, kapasitasMaksimal, hargaPerMalam, dan isTersedia dikunci menggunakan modifier private. Terdapat dua jenis *Constructor*: satu untuk pendaftaran kilat yang mengotomatiskan harga ke 0, dan satu lagi untuk pendaftaran lengkap.

Keamanan data diperkuat melalui validasi pada metode *Setter*, di mana setTipeKamar() hanya menerima kategori "Reguler", "Premium", atau "Suite" , sementara setHargaPerMalam() memaksa nilai minimal Rp50.000. Selain itu, diterapkan konsep Method Overloading pada fungsi pemesanan untuk mengecek kapasitas tamu , serta pada fungsi penagihan yang memberikan diskon 20% khusus untuk kode voucher "PROMO" dengan syarat menginap minimal 3 malam. Sebagai pelengkap, tersedia metode batalPesan() dan fitur untuk mencetak profil kamar secara rapi.

```java
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0;
        this.isTersedia = true;
    }

    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double harga) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(harga);
        this.isTersedia = true;
    }
```

Sistem ini menyediakan dua jenis **Constructor** untuk proses inisialisasi data. **Constructor pertama** ditujukan untuk pendaftaran kilat, di mana sistem hanya menerima tiga parameter (nomor, tipe, dan kapasitas) lalu secara otomatis mengatur harga ke 0.0 serta status ketersediaan menjadi true. Sementara itu, **Constructor kedua** melayani pendaftaran yang lebih lengkap dengan menerima empat parameter utama, termasuk harga per malam, sehingga pengguna bisa langsung menginput nilai harga sejak awal.

Hal menarik dalam proses inisiasi ini adalah penggunaan kata kunci this untuk merujuk langsung pada atribut objek. Selain itu, alih-alih hanya memasukkan nilai secara mentah, kedua *constructor* tersebut juga memanggil fungsi **Setter** (seperti setTipeKamar dan setHargaPerMalam). Tujuannya adalah agar setiap data yang masuk wajib melewati tahap validasi atau pengecekan bisnis terlebih dahulu sebelum benar-benar disimpan ke dalam sistem.

```java
public String getNomorKamar() { return nomorKamar; }
public String getTipeKamar() { return tipeKamar; }
public int getKapasitasMaksimal() { return kapasitasMaksimal; }
public double getHargaPerMalam() { return hargaPerMalam; }
public boolean isTersedia() { return isTersedia; }

public void setTipeKamar(String tipe) {
    if (tipe.equalsIgnoreCase("Reguler") || tipe.equalsIgnoreCase("Premium") || tipe.equalsIgnoreCase("Suite")) {
        this.tipeKamar = tipe;
    } else {
        System.out.println("Peringatan: Tipe '" + tipe + "' tidak valid. Otomatis diatur ke Reguler.");
        this.tipeKamar = "Reguler";
    }
}
```

Pada bagian ini, kita melakukan inisialisasi variabel satu per satu dengan menerapkan kontrol akses yang ketat. Untuk variabel nomorKamar, kita menyediakan metode *Getter* agar nilai yang sudah dienkapsulasi tetap bisa dibaca, serta *Setter* untuk memodifikasi nilainya secara standar.

Hal serupa dilakukan pada variabel tipeKamar, di mana metode *Getter* digunakan untuk mengambil data, sementara metode *Setter* dirancang dengan logika validasi bisnis yang lebih kompleks. Di dalam fungsi setTipeKamar(), sistem hanya akan menerima input kategori yang sah, yaitu "Reguler", "Premium", atau "Suite", tanpa mempedulikan penggunaan huruf besar maupun kecil. Jika ada input di luar kategori tersebut, sistem secara otomatis akan menolak data tersebut, mencetak peringatan, dan memaksa nilainya kembali ke kategori "Reguler" demi menjaga integritas data.

```java
public void setHargaPerMalam(double harga) {
    if (harga < 50000) {
        System.out.println("Peringatan: Harga di bawah 50000 ditolak. Otomatis diatur ke 50000.");
        this.hargaPerMalam = 50000;
    } else {
        this.hargaPerMalam = harga;
    }
}

public void batalPesan() {
    this.isTersedia = true;
}
```

Tahap berikutnya mencakup pengaturan variabel kapasitasMaksimal, hargaPerMalam, dan isTersedia dengan penjelasan sebagai berikut:

Selanjutnya, kita mengatur variabel kapasitasMaksimal menggunakan metode *Getter* untuk mengambil nilai yang telah dienkapsulasi, serta *Setter* untuk memodifikasi batas kapasitas secara standar. Untuk variabel hargaPerMalam, sistem menerapkan validasi bisnis yang sangat ketat melalui metode *Setter*. Modifikasi data tidak dilakukan secara bebas; sistem hanya menerima harga minimal Rp50.000. Jika pengguna mencoba memasukkan nominal di bawah angka tersebut, sistem akan otomatis menolak input, memberikan peringatan, dan memaksa nilai harga menjadi Rp50.000.

Variabel terakhir yang diatur adalah isTersedia, di mana metode *Getter* berfungsi untuk membaca dan mengembalikan status ketersediaan kamar saat ini, apakah bernilai true (tersedia) atau false (terisi). Selain itu, terdapat metode batalPesan() yang dirancang untuk membatalkan reservasi tamu. Saat metode ini dieksekusi, status isTersedia akan diubah kembali menjadi true, yang kemudian diikuti dengan pencetakan informasi ke layar bahwa kamar tersebut kini telah kosong dan siap dipesan kembali.

```java
public void pesanKamar() {
    if (isTersedia) {
        isTersedia = false;
        System.out.println("Kamar " + nomorKamar + " berhasil dipesan.");
    } else {
        System.out.println("Peringatan: Kamar " + nomorKamar + " sudah terisi.");
    }
}

public void pesanKamar(int jumlahTamu) {
    if (!isTersedia) {
        System.out.println("Peringatan: Kamar sudah terisi.");
    } else if (jumlahTamu > kapasitasMaksimal) {
        System.out.println("Ditolak: Jumlah tamu (" + jumlahTamu + ") melebihi kapasitas (" + kapasitasMaksimal + ")!");
    } else {
        isTersedia = false;
        System.out.println("Pemesanan berhasil untuk " + jumlahTamu + " orang.");
    }
}
```

Implementasi fitur pemesanan dalam kode ini memanfaatkan konsep *Method Overloading*, yang memungkinkan adanya dua mekanisme reservasi yang berbeda. Mekanisme pertama adalah metode pesanKamar() tanpa parameter yang berfungsi sebagai pemesanan instan; sistem hanya akan memvalidasi ketersediaan kamar, dan jika kosong, statusnya akan langsung diubah menjadi false atau terisi.

Mekanisme kedua, yaitu pesanKamar(int jumlahTamu), menerapkan prosedur validasi yang jauh lebih ketat. Selain mengecek ketersediaan, sistem secara otomatis akan membandingkan jumlah tamu dengan kapasitas maksimal kamar tersebut. Jika jumlah tamu masih sesuai batas, pesanan dinyatakan berhasil, namun jika terjadi kelebihan beban (*overcapacity*), sistem akan memblokir pesanan secara otomatis, memberikan pesan teguran khusus, dan tetap mempertahankan status kamar sebagai tersedia (true).

```java
public double hitungTotalBayar(int jumlahMalam) {
    return hargaPerMalam * jumlahMalam;
}

public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
    double total = hitungTotalBayar(jumlahMalam);
    if (kodeVoucher.equalsIgnoreCase("PROMO") && jumlahMalam >= 3) {
        System.out.println("Voucher Berhasil: Diskon 20% diterapkan.");
        return total * 0.8;
    } else {
        System.out.println("Voucher Gagal: Syarat tidak terpenuhi atau kode salah. Harga normal diterapkan.");
        return total;
    }
}
```

Bagian terakhir dari sistem ini adalah mekanisme penagihan yang juga menerapkan konsep *Method Overloading* untuk menangani dua skenario pembayaran berbeda.

Dalam skenario pertama, yaitu pembayaran standar tanpa voucher, sistem menggunakan metode hitungTotalBayar(int jumlahMalam) yang perhitungannya cukup sederhana, yakni mengalikan hargaPerMalam dengan durasi menginap.

Sedangkan untuk skenario kedua, yaitu pembayaran dengan kode voucher, sistem menggunakan metode hitungTotalBayar(int jumlahMalam, String kodeVoucher) yang memiliki logika lebih kompleks. Pertama, sistem akan menghitung tagihan normal dan menyimpannya sebagai dasar perhitungan. Selanjutnya, sistem melakukan seleksi ketat terhadap dua syarat utama: kode voucher harus berupa "PROMO" dan durasi menginap minimal 3 malam.

Apabila salah satu syarat tidak terpenuhi, sistem akan memberikan notifikasi kegagalan penggunaan voucher dan tetap menagihkan harga normal. Namun, jika semua syarat terpenuhi, sistem akan menghitung diskon sebesar 20% (atau $totalNormal \times 0.20$) dan mengurangkannya dari total tagihan normal untuk mendapatkan nilai akhir yang harus dibayar oleh tamu.

```java
    public void tampilkanProfil() {
        System.out.println("=== Profil Kamar " + nomorKamar + " ===");
        System.out.println("Tipe: " + tipeKamar);
        System.out.println("Kapasitas: " + kapasitasMaksimal + " orang");
        System.out.println("Harga/Malam: Rp" + hargaPerMalam);
        System.out.println("Status: " + (isTersedia ? "Tersedia" : "Dipesan"));
        System.out.println("---------------------------");
    }
}
```

Terakhir, sistem ini dilengkapi dengan metode tampilkanProfilKamar() yang berfungsi layaknya "mesin cetak struk" untuk menyajikan informasi secara visual. Metode ini didefinisikan dengan *keyword* void karena murni bertugas melakukan aksi tanpa mengembalikan nilai perhitungan apa pun ke dalam sistem.

Cara kerjanya sangat praktis: metode ini menggunakan perintah System.out.println untuk menyusun dan menampilkan seluruh data krusial kamar secara sistematis. Informasi yang ditampilkan mencakup nomor kamar, tipe, kapasitas maksimal, hingga harga per malam, sehingga staf hotel dapat membaca kondisi akhir setiap objek kamar dengan rapi dan mudah.

MAIN :

```java
public class Main {
    Run | Debug
    public static void main(String[] args) {
        KamarHotel kamar1 = new KamarHotel(nomorKamar: "101", tipeKamar: "Presidential", kapasitasMaksimal: 2, -10000);
        KamarHotel kamar2 = new KamarHotel(nomorKamar: "102", tipeKamar: "Premium", kapasitasMaksimal: 2);
        kamar2.setHargaPerMalam(harga: 100000);

        System.out.println("\n--- Simulasi Operasional ---");
        System.out.println("Tagihan Kamar 1 (2 malam, PROMO): " + kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO"));
        kamar2.pesanKamar(jumlahTamu: 4);
        kamar2.pesanKamar(jumlahTamu: 2);
        kamar2.pesanKamar();
        System.out.println("Tagihan Kamar 2 (4 malam, PROMO): " + kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO"));

        System.out.println("\n--- Struk Informasi Akhir ---");
        kamar1.tampilkanProfil();
        kamar2.tampilkanProfil();
    }
}
```

Bagian terakhir adalah kelas Main yang berfungsi sebagai arena pembuktian untuk menguji ketahanan sistem yang telah kita bangun. Di sini, kita mensimulasikan berbagai skenario operasional untuk memastikan bahwa seluruh validasi dan logika bisnis berjalan dengan semestinya. Pertama, kita membuat dua objek kamar menggunakan jenis *constructor* yang berbeda. Pada objek kamar1, kita sengaja menguji ketahanan metode *Setter* dengan memasukkan data yang tidak valid, yaitu tipe kamar "Presidential" dan harga "-10000". Hasilnya, sistem secara otomatis akan mengoreksi data tersebut menjadi "Reguler" dan harga minimal "50000" sesuai dengan aturan bisnis yang telah ditetapkan.

Selanjutnya, kita melakukan serangkaian uji coba pada objek kamar2 untuk membuktikan fungsionalitas *Method Overloading* dan logika ketersediaan. Kita mencoba memesan kamar untuk 4 orang padahal kapasitas maksimalnya hanya 2, dan sistem berhasil memblokirnya secara otomatis karena terjadi *overcapacity*. Setelah itu, kita melakukan pemesanan ulang dengan jumlah tamu yang sesuai (2 orang) hingga statusnya dinyatakan berhasil. Saat kita mencoba memanggil kembali metode pesanKamar() tanpa parameter pada kamar yang sama, sistem secara cerdas akan mengeluarkan peringatan bahwa kamar tersebut sudah terisi atau sudah dipesan.

Terakhir, kita mendemonstrasikan fitur penagihan cerdas dengan menggunakan voucher "PROMO". Pada kamar1, simulasi menunjukkan bahwa perhitungan tagihan untuk 2 malam menggunakan voucher tersebut gagal mendapatkan diskon karena tidak memenuhi syarat minimal 3 malam. Sebaliknya, pada kamar2, perhitungan tagihan untuk 4 malam dengan voucher "PROMO" berhasil mendapatkan potongan diskon sebesar 20%. Simulasi ini ditutup dengan mencetak "Struk Informasi" yang rapi ke layar untuk menampilkan seluruh kondisi akhir dari atribut kedua objek kamar tersebut secara transparan.

IV. Kesimpulan

Secara keseluruhan, proyek KamarHotel ini adalah contoh penerapan nyata yang sangat baik dari prinsip Object-Oriented Programming (OOP) untuk membangun sistem operasional yang cerdas dan tahan terhadap bug. Melalui penerapan Enkapsulasi, kita berhasil menciptakan tameng data dengan menyembunyikan atribut penting dan memasang validasi ketat pada Setter, sehingga sistem mustahil disusupi oleh kesalahan input Selain itu, pemanfaatan Method Overloading menjadikan sistem ini sangat adaptif, ia mampu menangani berbagai skenario resepsionis di lapangan secara fleksibel, mulai dari pemesanan kilat, pencegahan overcapacity, hingga perhitungan tagihan bersyarat menggunakan voucher promo.